



JUAN CARLOS ROMERO GIL



UNIVERSIDAD SANTO TOMÁS
DIVISIÓN DE EDUCACIÓN ABIERTA Y A DISTANCIA
FACULTAD DE CIENCIAS Y TECNOLOGÍAS
INGENIERIA INFORMATICA
TUNJA
2026-2

INFORME TÉCNICO

Estudiante

JUAN CARLOS ROMERO GIL

Integración de Sistemas de Información

Docente

Crisman Martínez Barrera

UNIVERSIDAD SANTO TOMÁS
DIVISIÓN DE EDUCACIÓN ABIERTA Y A DISTANCIA
FACULTAD DE CIENCIAS Y TECNOLOGÍAS
INGENIERIA INFORMATICA
TUNJA
2026-2

Tabla de contenido

Sistema de Integración Hardware, Software y Firmware (HSF).....	4
Introducción.....	4
1. Objetivos	4
2. Caso de uso y datos de entrada.....	4
3. Arquitectura e integración	5
4. Organización del software.....	5
5. Persistencia, reporte y observabilidad	6
6. Validación, anomalías y manejo de errores.....	6
7. Matriz de pruebas	7
8. Seguridad, rendimiento y escalabilidad.....	8
8.1. Seguridad.....	8
8.2. Rendimiento	8
8.3. Escalabilidad	8
9. Respuestas a las preguntas técnicas del Anexo 3	8
Conclusiones.....	11
Referencias	12

Sistema de Integración Hardware, Software y Firmware (HSF)

Introducción

Este informe documenta el diseño de un prototipo de integración entre hardware simulado, firmware/emulador, sistema operativo y una aplicación desarrollada en Python.

El caso de uso consiste en monitorear un dispositivo IoT identificado como SENSOR-001, que genera telemetría con temperatura, humedad, voltaje, estado operativo y fecha/hora.

La solución busca ilustrar el recorrido de los datos desde su generación hasta su validación, clasificación, persistencia y presentación en archivos de salida.

Según la presentación del proyecto, los artefactos de salida son telemetria.csv, reporte.json y sistema.log.

Alcance documental: el informe se contrastó con los archivos de código, README, matriz de pruebas, CSV, JSON y log incluidos en el archivo tarea vect.rar.

Los resultados que se reportan corresponden a los artefactos entregados; se señalan expresamente las discrepancias entre ellos.

1. Objetivos

- Describir la arquitectura de integración HSF del prototipo SENSOR-001.
- Explicar las responsabilidades de la fuente simulada, el emulador, el sistema operativo y la aplicación Python.
- Documentar la validación y clasificación de telemetría, así como la persistencia y la observabilidad previstas.
- Definir pruebas verificables y registrar de forma transparente los resultados que deben confirmarse con evidencias.
- Identificar medidas de seguridad, confiabilidad, rendimiento y escalabilidad.
-

2. Caso de uso y datos de entrada

El prototipo representa un sistema de monitoreo de un dispositivo IoT. En lugar de una placa física, la presentación indica el uso de un programa Python que simula el comportamiento del dispositivo.

Elemento	Descripción documentada
Identificador	SENSOR-001
Temperatura	Temperatura registrada por la fuente simulada.
Humedad	Porcentaje de humedad.
Voltaje	Voltaje reportado por el dispositivo simulado.

Estado	Estado operativo del dispositivo.
Fecha/hora	Marca temporal de la medición.
Regla de anomalía	La presentación clasifica como anomalía una temperatura superior a 50 °C.

La validación debe comprobar la presencia de los campos requeridos, los tipos de datos y los rangos admitidos. Los rangos específicos de humedad y voltaje, además de la frecuencia exacta de lectura, deben documentarse a partir del código real; no se fijan aquí porque no están especificados en los materiales consultados.

3. Arquitectura e integración

Capa	Responsabilidad	Evidencia/artefacto indicado
Hardware simulado	Representar la fuente física de señales y mediciones.	Datos generados por el simulador.
Firmware/emulador	Simular el comportamiento del dispositivo y generar telemetría estructurada.	Trama o registro de telemetría.
Comunicación	Transportar los datos entre el emulador y la aplicación.	Interfaz de comunicación definida en el código; confirmar protocolo real.
Sistema operativo	Administrar procesos, archivos, memoria y otros recursos usados por Python.	Consulta del sistema operativo y de procesos activos.
Aplicación Python	Adquirir, validar, clasificar y procesar la telemetría.	Módulos de adquisición, validación/procesamiento y coordinación.
Persistencia y observabilidad	Guardar mediciones, resumen de ejecución y eventos.	telemetry.csv, reporte.json y sistema.log.

Flujo lógico documentado: hardware simulado → firmware/emulador → comunicación → sistema operativo → aplicación Python → validación y procesamiento → CSV/JSON/log → usuario.

La presentación señala que la aplicación consulta en Windows el sistema operativo, la versión del sistema y los procesos activos.

La implementación concreta de estas consultas debe verificarse en el código fuente.

La documentación de esta sección debe actualizarse si el código utiliza otros recursos o un protocolo diferente.

4. Organización del software

La presentación describe una aplicación modular con responsabilidades diferenciadas:

- Adquisición: recibe la telemetría de la fuente simulada.
- Validación y procesamiento: verifica campos, tipos y rangos, y clasifica los estados normales y anómalos.
- Integración con el sistema operativo: consulta información del entorno y recursos del equipo.
- Coordinación principal (main): organiza la ejecución, registra eventos y genera los archivos de salida.

Los módulos verificados son `app/adquisicion.py` (clase `Adquisicion`), `app/procesamiento.py` (clase `Procesamiento`), `app/sistema.py` (clase `Sistema`) y `app/main.py` (coordinación). La fuente `firmware/simulador.py` genera telemetría en un proceso separado; el código de `app/adquisicion.py` vuelve a simular la recepción por su cuenta, por lo que no se observa un canal real entre ambos procesos.

5. Persistencia, reporte y observabilidad

Artefacto	Propósito indicado	Comprobación necesaria
telemetry.csv	Almacenar las mediciones recibidas.	Verificar encabezados, filas, tipos, marcas temporales y tratamiento de datos inválidos.
reporte.json	Resumir la ejecución.	Confirmar las métricas y campos reales del JSON generado.
sistema.log	Registrar eventos, anomalías y errores.	Confirmar fecha/hora, nivel, mensaje, errores y continuidad de la ejecución.

En el reporte entregado figuran 15 ciclos, 14 datos válidos, 4 anomalías, 1 error y 93,33 % de datos válidos.

Estos valores describen el artefacto guardado; la matriz de pruebas, sin embargo, declara 3 anomalías y 1 error, diferencia que debe corregirse antes de la entrega.

6. Validación, anomalías y manejo de errores

La regla de negocio explícita en la presentación es clasificar como anomalía una temperatura superior a 50 °C. Además, la solución debe validar la estructura, el tipo y el rango de los datos recibidos.

- Si faltan campos o hay tipos incorrectos, el registro debe rechazarse o marcarse como inválido según la política implementada.
- Ante pérdida de comunicación o dispositivo no disponible, la aplicación debe detectar el problema, registrar el evento y evitar un cierre inesperado.
- Ante una ruta de salida inaccesible, se debe registrar el error y comunicar que no fue posible persistir el dato; no debe afirmarse recuperación automática hasta comprobarla.
- Los datos externos deben tratarse como no confiables. No se deben ejecutar comandos del sistema derivados directamente de la telemetría.

Las respuestas anteriores distinguen las capacidades verificadas en el código de las recomendaciones de diseño. Las pruebas y artefactos incluidos respaldan la clasificación de

anomalías, la persistencia y el registro de eventos; las pruebas de permisos y la recuperación automática ante cualquier fallo no están demostradas por los archivos revisados.

7. Matriz de pruebas

La matriz original del proyecto marca ocho pruebas como aprobadas. Se contrastaron también los archivos de salida incluidos en el ZIP. Las diferencias de conteo se documentan después de la tabla para mantener trazabilidad.

ID	Prueba/condición	Resultado esperado	Resultado real / estado
TP-01	Telemetría válida dentro de rango.	Aceptar y procesar el registro; persistirlo.	La telemetría.csv contiene registros normales. Aprobada según matriz.
TP-02	Temperatura superior a 50 °C.	Clasificar el registro como anomalía.	CSV y reporte.json muestran 4 anomalías; matriz dice 3. Aprobada con discrepancia.
TP-03	Trama/dato con formato incorrecto.	Rechazar el dato y continuar.	El log registra un dato rechazado por ausencia de timestamp; la matriz declara que el proceso continúa. Aprobada según matriz.
TP-04	Pérdida de comunicación.	Controlar excepción y continuar.	La clase Adquisicion puede lanzar ConnectionError y main.py la captura; el log entregado no muestra este evento. Aprobada según matriz, evidencia de ejecución no visible en este log.
TP-05	Consulta del sistema operativo.	Obtener sistema, versión y procesos.	app/sistema.py consulta platform y ejecuta tasklist; matriz reporta Windows. Aprobada según matriz.
TP-06	Persistencia CSV.	Guardar mediciones procesadas.	telemetria.csv contiene encabezado y 14 registros aceptados. Aprobada.
TP-07	Reporte JSON.	Generar resumen de ejecución.	reporte.json contiene 15 ciclos, 14 válidos, 4 anomalías, 1 error y 93,33

			% Aprobada con discrepancia frente a matriz.
TP-08	Registro de eventos.	Registrar inicio, normalidad, anomalías y errores.	sistema.log incluye inicio/fin, rechazo de dato, registros normales y anomalías. Aprobada.

Observación de consistencia: reporte.json y telemetria.csv coinciden en cuatro anomalías y catorce registros aceptados, mientras matriz_pruebas.md informa tres anomalías. El log muestra un rechazo por ausencia de timestamp, aunque el código de adquisición genera también fallos de comunicación y tramas corruptas aleatorias. El archivo de log entregado no contiene un evento de pérdida de comunicación en esa ejecución. Se debe unificar la matriz con el reporte real que se decida presentar.

8. Seguridad, rendimiento y escalabilidad

8.1. Seguridad

Se recomienda validar entradas antes de procesarlas, limitar los permisos de escritura a las rutas necesarias, evitar la ejecución de comandos contruidos a partir de datos recibidos y capturar excepciones de manera controlada. Las credenciales o secretos no deben incluirse en el repositorio.

8.2. Rendimiento

No se incluye una medición de latencia de comunicación porque la aplicación no mide ese indicador y la fuente de telemetría está simulada localmente.

8.3. Escalabilidad

Para incorporar 100 dispositivos, la arquitectura podría evolucionar hacia adquisición concurrente, identificación inequívoca de cada dispositivo, colas o intermediarios de mensajería, almacenamiento centralizado, límites de recursos, supervisión de conexiones y políticas de reintento. Esto es una propuesta de evolución, no una característica demostrada por el prototipo actual.

9. Respuestas a las preguntas técnicas del Anexo 3

¿Qué responsabilidad pertenece al hardware y cuál al firmware? ¿Dónde se transforma cada dato? El hardware representa los componentes físicos que capturan o generan señales; el firmware controla el dispositivo y su protocolo. En esta práctica la fuente es simulada: la telemetría la genera el emulador y la aplicación Python valida y clasifica los registros.

¿Qué servicios del sistema operativo utiliza Python? La presentación menciona la consulta del sistema operativo, su versión y los procesos activos en Windows. Los módulos y llamadas exactos deben confirmarse en el código.

¿Qué ocurre si el sistema operativo revoca permisos? La aplicación debe manejar el error de acceso, registrarlo y evitar terminar inesperadamente. La capacidad real de continuar depende de dónde ocurra el fallo y debe probarse.

¿Qué ventajas y limitaciones tiene Python frente a C/C++ o Rust? Python facilita prototipado, legibilidad y pruebas, pero normalmente ofrece menos control de bajo nivel y mayor sobrecarga de ejecución. C/C++ y Rust pueden resultar apropiados para componentes con restricciones de recursos o tiempo real estricto.

¿Cómo cambiaría la arquitectura para 100 dispositivos? Se requeriría gestionar múltiples conexiones concurrentes, identificación por dispositivo, colas o mensajería, persistencia centralizada y supervisión de carga.

¿Qué riesgos aparecen al recibir datos externos o ejecutar comandos? Datos malformados, valores fuera de rango, inyección de comandos, acceso no autorizado y exposición de información. La mitigación incluye validación, privilegio mínimo y evitar comandos derivados directamente de la entrada.

¿Cómo garantizar integridad y trazabilidad? Validar y registrar marcas temporales, identificadores, resultado de validación y errores; restringir permisos de archivos y conservar registros consistentes. Las garantías específicas dependen de la implementación.

¿Qué diferencias puede haber entre Windows y Linux? Cambian rutas, permisos, nombres y herramientas de procesos, servicios y puertos. La portabilidad exige aislar dependencias del sistema y documentar los comportamientos específicos.

¿Cómo actualizar firmware sin afectar el servicio? En una evolución con dispositivos reales: versionar las actualizaciones, validar integridad y autenticidad, desplegar gradualmente, comprobar salud y disponer de reversión. No aplica como capacidad demostrada del emulador actual.

¿Qué pruebas adicionales aplicar antes de producción? Pruebas de carga, duración, entradas límite y malformadas, desconexión prolongada, recuperación, permisos, seguridad, compatibilidad y restauración de datos.

¿Cuál sería el principal punto de falla? La fuente de datos o el canal de comunicación pueden interrumpir la cadena. Se recomienda detectar desconexiones, aplicar tiempos de espera y reintentos controlados, registrar eventos y definir qué hacer cuando el servicio no se recupera.

Conclusiones

El prototipo entregado implementa una fuente simulada SENSOR-001 y una aplicación Python modular que valida campos y tipos, controla rangos de humedad y voltaje, clasifica temperaturas superiores a 50 °C como anomalías y genera CSV, JSON y logs. En los artefactos entregados, la ejecución reporta 15 ciclos, 14 datos válidos, cuatro anomalías, un error y un 93,33 % de datos válidos.

La separación de responsabilidades facilita el mantenimiento, pero la integración actual es principalmente lógica/simulada: firmware/simulador.py imprime telemetría en su propio bucle y app/adquisicion.py genera otra telemetría independiente. No se encontró una conexión por puerto serial, socket, HTTP o MQTT entre ambos procesos. Para afirmar comunicación efectiva entre firmware y aplicación sería necesario implementar ese canal o describir con precisión la emulación.

Antes de entregar, se recomienda corregir la discrepancia entre la matriz (3 anomalías) y reporte.json/telemetry.csv (4 anomalías), y confirmar si se desea describir la fuente como emulación local o implementar comunicación real entre firmware y aplicación. Los artefactos incluidos sirven como evidencia, aunque no sustituyen capturas del video si el docente las exige.

Referencias

Martínez Barrera, C. (2026). Evaluación segundo corte: Integración de Sistemas de Información 2026-2 [Guía de evaluación]. Universidad Santo Tomás.

Python Software Foundation. (s. f.). Python documentation. <https://docs.python.org/3/>

Universidad Santo Tomás. (2026). Sistema de Integración Hardware, Software y Firmware (HSF): Práctica de integración [Presentación de clase proporcionada por el estudiante].